INFO 4220 – Networks II: Market Design Spring 2025 – Assignment 1

Due date: (Thursday) March 20, 11.59PM EST

Late submissions will be penalized with 0.25 point for every day late. No submissions accepted after 1 week.

Instructions:

- You can work with up to 1 person
- Create a document with your answer. Include your code (with a short explanation) and the output of your code, along with the response to the prompts below
- You can use GenAI to help you code. This is not a coding assignment (although it requires coding). Instead, I am interested in how you interpret the results of your simulations and how you relate it with what we have covered in the course. You still need to provide your code and output or your responses will not be considered valid

Part 1: Setup and tutorial (5 points)

For this part you will setup R in your computer and numerically compute a random assignment. If you don't have R currently installed, a simple way of quickly setting up a usable environment is installing R Studio. To do so follow the steps below:

1. If you don't have R installed in your computer, download and install R and R studio: https://posit.co/download/rstudio-desktop/#download
2. If you don't have java installed, also install java: https://www.java.com/en/download/

After installing R, follow the steps below to numerically compute a random serial dictatorship

```
# Install and load the matchingMarkets library. If you want to look at
the documentation of the library for more info, look at:
https://cran.r-
project.org/web/packages/matchingMarkets/matchingMarkets.pdf

install.packages("matchingMarkets")
library(matchingMarkets)
# Create vectors with the preferences of Alice, Bob, Carol, and Denis
(Note: This is the example in the slides of L8: Probabilistic
```

```
Assignments. Object 1 is Apple, 2 is Orange, 3 is Pear, and 4 is
Strawberry)
alice<-c(1,2,3,4)
bob<-c(1,2,3,4)
carol<-c(2,1,4,3)
denis<-c(2,1,4,3)

# Combine the preferences into a matrix. Note that the way the
matchingMarkets library wants the matrix is different than what we saw
in class (the library wants people in columns and objects in rows)

matrix_prefs<-matrix(c(alice, bob, carol, denis), 4,4)

# Next, we create a vector with the four dictarors (we will use the
later for drawing random orders)

dictators <- c(1,2,3,4)

# Then we create an empty 4x4 matrix to store the results
results <- matrix(0, 4, 4)

# This vector will tell the rsd command that we have 1 of each object
nSlots<-c(1,1,1,1)

# The following vector contains the first preference (i.e. the most
preferred object) for each individual. What I am doing below is taking
the first row of the matrix with individuals' preferences and storing
it as vector (column) called first_prefs. I also create a variable
called count_first to count how many individuals got their most
preferred object. I will use these to keep track of the how many times
individuals are getting their most preferred object in my simulations

first_prefs <- matrix(c(matrix_prefs[1,]))
count_first <- 0

# Then repeat the same for second, third, and fourth preferences
second_prefs <- matrix(c(matrix_prefs[2,]))
third_prefs <- matrix(c(matrix_prefs[3,]))
fourth_prefs <- matrix(c(matrix_prefs[4,]))
count_second <- 0
count_third <- 0
count_fourth <- 0

# The loop below runs 10,000 random serial dictatorships and stores
the results in the matrix results we created above. It also counts how
many times individuals got their most preferred object, their second
most preferred object, etc.

for (i in 1:10000){
     # We first draw a random order of dictators
     order <- sample(dictators)
```

```
     # Then we run the serial dictatorship. This will return a matrix
with the results. The first column is the list of individuals, and the
second column is the objects they were assigned to
     assignment <- rsd(prefs = matrix_prefs, priority = order, nSlots
= nSlots)

     # The loop below updates the results matrix. It adds one to the
(object, person) cell if the person was assigned to the object
     for (i in 1:4){
          results[i, assignment[i,2]] <- results[i,
assignment[i,2]]+1
          }

     # The command below is going to compare the vector of first
preferences with the second column of the assignment matrix (which
contains the assignment). It will create a vector that contains a 1 if
person i was assigned to their most preferred object
     first_choice <- matrix(c(ifelse(first_prefs == assignment[,2], 1,
0)))


     # Then we just update the counter that counts how many times
individuals have been assigned their most preferred object
     count_first <- count_first + sum(first_choice)

     # We repeat this for the second, third, and fourth most preferred
objects
     second_choice <- matrix(c(ifelse(second_prefs == assignment[,2],
1, 0)))
     count_second <- count_second + sum(second_choice)
     third_choice <- matrix(c(ifelse(third_prefs == assignment[,2], 1,
0)))
     count_third <- count_third + sum(third_choice)
     fourth_choice <- matrix(c(ifelse(fourth_prefs == assignment[,2],
1, 0)))
     count_fourth <- count_fourth + sum(fourth_choice)
     }

# Finally we divide our results variable by 10,000 to express results
in terms of probabilities, and the counters by 10,000 to express them
as how many individuals on average get their most preferred object,
their second most preferred object, etc.

results <- results/10000
count_first <- count_first/10000
count_second <- count_second/10000
count_third <- count_third/10000
count_fourth <- count_fourth/10000
```

Now answer the following questions:

1. Compare your results matrix with the probabilistic assignment we computed in L8. Are the results similar? (1 point)
2. In your simulations, on average, how many people obtain their most preferred object, one of their 2 most preferred objects, or one of their 3 most preferred objects? (2 points)
3. If instead of using a serial random dictatorship you used the probabilistic assignment produced by the eating algorithm and did a similar simulation. How many people obtain their most preferred object, one of their 2 most preferred objects, or one of their 3 most preferred objects? (2 points)

Hint for 3: Use the assignment that results from using the eating algorithm (it is in the lecture slides; you can use it directly). Then, decompose it as a lottery over deterministic assignments. From there, it should be straightforward to infer what would happen if you randomly chose 1 of the deterministic assignments randomly many times (you can simulate with R if you want, but it is also OK if you just reason and explain what would happen).

Part 2: Simulating a school assignment problem (25 points)

You have been hired by a school district in Central New York to decide which algorithm to use for their new school choice program. The school district and the community cannot decide whether to use deferred acceptance or top trading cycle. For the next town hall meeting you have been tasked with producing a simulation that exemplifies the outcomes that would result from using each algorithm. To do so, follow the following assumptions/steps:

- There are 4 schools with capacities 10, 20, 15, 15
- There are 60 students that need to be assigned to schools
- For each school, create a random order of students (one different random order for each school)
- Simulate 5000 matchings. For each iteration:
    - Start by creating a random order of student preferences over school (for each iteration create new student preferences while keeping school priorities fixed)
    - Create the assignment that results from running DA. Count how many students get their first, second, or third preference.

    - Create the assignment that results from running TTC. Count how many students get their first, second, or third preference.
- At the end of the 5000 simulations, report how many students get their first, second, and third preference on average with each of the algorithms

Answer the following questions:

1. If the objective is to maximize the number of students that get their favorite schools, which algorithm would you recommend? (15 points)
2. Is there any other reason why you would recommend using TTC? (5 points)
3. Is there any other reason why you would recommend using DA? (5 points)

Hints for using matchingMarkets in R

Example (how to run TTC with matchingMarkets in R). Let's say we have 5 students and 2 schools with capacities 2 and 3.

```
install.packages("matchingMarkets ")
library(matchingMarkets)
schools <- matrix(c(1, 2))
nSlots <- c(2, 3)
students <- matrix(1:5, 5, 1)
school_priorities <- matrix(0, 5, 2)
for (i in 1:2){
     school_priorities[,i] <- sample(students)
     }
student_preferences <- matrix(0, 2, 5)
for (i in 1:5){
     student_preferences[,i] <- sample(schools)
     }
resttc <- ttc2(s.prefs = student_preferences, c.prefs =
school_priorities, nSlots = nSlots)
```

After running this, resttc will be a matrix with the result in which the first column will list the individuals, and the second column the assignments.

Example (how to run DA with matchingMarkets in R). Let's say we have 5 students and 2 schools with capacities 2 and 3.

```
install.packages("matchingMarkets")
library(matchingMarkets)
schools <- matrix(c(1, 2))
nSlots <- c(2, 3)
students <- matrix(1:5, 5, 1)
```

```
school_priorities <- matrix(0, 5, 2)
for (i in 1:2){
     school_priorities[,i] <- sample(students)
     }
student_preferences <- matrix(0, 2, 5)
for (i in 1:5){
     student_preferences[,i] <- sample(schools)
     }
resda <- hri(s.prefs = student_preferences, c.prefs =
school_priorities, nSlots = nSlots)
```

After running this, resda will be more complicated as it is a dataframe that contains multiple things. You are interested on the matchings results. Below is an example of the output I got (you can access this matrix as: resda[["matchings "]]:

```
$matchings
  matching college slots student sOptimal cOptimal sRank cRank
1        1       1     2       1        1        1     2     5
2        1       1     1       4        1        1     1     1
3        1       2     5       2        1        1     3     4
4        1       2     3       3        1        1     1     2
5        1       2     4       5        1        1     2     3
```

Columns college and student tells you who was matched to which school. Column sOptimal tells you if the matching is the Student Optimal matching, and cOptimal tells you if it is the college optimal matching. In this case, there is only one stable matching, so the assignment is both student optimal and school optimal, but in cases where they are different, both results will be listed, and you would distinguish them using those columns. Below are some operations you can use to re-arrange the data in a way that is easier to use:

```
# Keep only the matching results
resda_matchings <- resda[["matchings"]]
# We are only interested in the student optimal matching, so we delete
results that are not student optimal
resda_matchings <- subset(resda_matchings, sOptimal == 1)
In the results matrix numbers are stored as strings rather than
characters. To make them easy to sort, lets convert to integers
resda_matchings$student <- as.integer(resda_matchings$student)
resda_matchings$college <- as.integer(resda_matchings$college)
# Let's sort the results by student
resda_matchings <- resda_matchings[order(resda_matchings$student),]
# Now your student assignments can be seen (sorted from students 1 to
5):
matrix(c(resda_matchings$college))
```